

Computer Science I

Week 4

Tutorial Session

Exercise 1 - Sum The Elements Of An Array

In `exercises/exercise_1.c`, complete the function:

```
int sum_array(int a[], int n);
```

The function must return the sum of the first `n` elements of `a`. If `n == 0`, it must return 0. You may assume that all sums fit in an `int`.

For example:

```
a = [3, -1, 4, 2]
sum_array(a, 4) returns 8
```

The file already contains a `main` function that tests your solution.

Exercise 2 - Count With Pointer Arithmetic

In `exercises/exercise_2.c`, complete the function:

```
int count_value(int *a, int n, int target);
```

The function must return the number of times `target` appears among the first `n` elements of `a`. Inside the function, do not use the array indexing operator `[]`. Move through the array using a pointer and pointer arithmetic.

For example:

```
a = [2, 5, 2, 8, 2]
count_value(a, 5, 2) returns 3
```

Remember that `a + n` points one position past the final element. For example, a loop can compare its current pointer with `a + n` to decide whether there are elements left to process. The pointer `a + n` marks the stopping position; it does not point to an array element and must not be dereferenced.

Exercise 3 - Translate A Point

Consider the following structure:

```
struct Point {
    double x;
    double y;
};
```

In `exercises/exercise_3.c`, complete these functions:

```
void translate(struct Point *p, double dx, double dy);
double squared_distance_from_origin(struct Point *p);
```

`translate` must add `dx` and `dy` to the coordinates of the point. `squared_distance_from_origin` must return the squared distance of the point from the origin.

Recall that when `p` points to a structure, `p->x` accesses its field `x`. It is a shorter way of writing `(*p).x`.

Starting from the point (2.0, -1.0), translating by (3.0, 4.0) produces (5.0, 3.0), whose squared distance from the origin is 34.0.

Exercise 4 - Allocate A Copy Of An Array

In `exercises/exercise_4.c`, complete the function:

```
int *copy_array(int *a, int n);
```

The function must:

- return `NULL` if `n <= 0`;
- allocate space for exactly `n` integers;
- copy the elements of `a` into the new array;
- return `NULL` if the allocation fails;
- otherwise return a pointer to the new array.

Include `stdlib.h` to use `malloc`, `free`, and `NULL`. The caller owns the returned array and must call `free` when it is no longer needed.

To allocate space for `n` integers and check whether the allocation failed, you can use:

```
int *copy;

copy = malloc(n * sizeof(int));
if (copy == NULL) {
    return NULL;
}
```

Changing an element of the copy must not change the original array. The supplied `main` function performs this test and frees the copy.

Exercise 5 - Extract Values In A Range

We are given an array `a` of length `n` and two integers `low` and `high`, with `low <= high`. A value is selected when it belongs to the closed interval `[low, high]`.

In `exercises/exercise_5.c`, complete these functions:

```
int count_in_range(int *a, int n, int low, int high);
int *extract_in_range(int *a, int n, int low, int high, int k);
```

`count_in_range` must return the number of selected values.

`extract_in_range` receives this number as `k`. If `k == 0`, it must return `NULL`. Otherwise, it must allocate an array of exactly `k` integers and copy all selected values into it, preserving their original order. If the allocation fails, return `NULL`.

For example:

```
a = [8, 3, 6, 1, 5, 9]
low = 3, high = 6
k = 3
result = [3, 6, 5]
```

The supplied main function tests both functions, prints the extracted array, and frees it.

Exercise 6 - Compute Array Statistics

Consider the following structure:

```
struct Statistics {
    int minimum;
    int maximum;
    double average;
};
```

In exercises/exercise_6.c, complete the function:

```
struct Statistics *summarize(int *a, int n);
```

If $n \leq 0$, return NULL. Otherwise, allocate one `struct Statistics` and initialize its fields with the minimum, maximum, and average of the array. Return NULL if the allocation fails.

To allocate one structure and check whether the allocation failed, you can use:

```
struct Statistics *result;

result = malloc(sizeof(struct Statistics));
if (result == NULL) {
    return NULL;
}
```

For example:

```
a = [4, 7, 1, 8]
minimum = 1
maximum = 8
average = 5.0
```

You may assume that the sum fits in an `int`. The caller is responsible for freeing the returned structure.

Exercise 7 - Extract Local Peaks

An element $a[i]$ is a *local peak* when it is strictly larger than both of its neighbours:

$$i > 0, \quad i < n - 1, \quad a[i] > a[i-1], \quad a[i] > a[i+1].$$

The first and last elements are never local peaks.

Use the following structure:

```
struct Peaks {
    int *values;
    int length;
};
```

In exercises/exercise_7.c, complete these functions:

```
int is_peak(int *a, int n, int i);
int count_peaks(int *a, int n);
struct Peaks *extract_peaks(int *a, int n);
```

`is_peak` must return 1 when position `i` contains a local peak and 0 otherwise. It must also return 0 when `i` is outside the array or refers to the first or last element.

`count_peaks` must count all local peaks.

If `n <= 0`, `extract_peaks` must return `NULL`. Otherwise, it must allocate a `struct Peaks`. Its `length` must contain the number of peaks. If there are peaks, its `values` field must point to a newly allocated array containing them in their original order. If there are no peaks, set `values` to `NULL` and `length` to 0.

For example:

```
a = [1, 5, 2, 7, 4, 3, 6, 1]
length = 3
values = [5, 7, 6]
```

If either required allocation fails, release any memory already allocated and return `NULL`. The caller must eventually free both `result->values` and `result`.